

The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems

Linke Song¹, Zixuan Pang, Wenhao Wang¹, Zihao Wang¹, XiaoFeng Wang, *Fellow, IEEE*, Hongbo Chen, Wei Song¹, *Senior Member, IEEE*, Yier Jin¹, *Senior Member, IEEE*, Dan Meng, and Rui Hou¹

Abstract—The wide deployment of Large Language Models (LLMs) has given rise to strong demands for optimizing their inference performance. Today’s techniques serving this purpose primarily focus on reducing latency and improving throughput through algorithmic and hardware enhancements, while largely overlooking their privacy side effects, particularly in a multi-user environment. In our research, for the first time, we discovered a set of new timing side channels in LLM systems, arising from shared caches and GPU memory allocations, which can be exploited to infer both confidential system prompts and those issued by other users. These vulnerabilities echo security challenges observed in traditional computing systems, highlighting an urgent need to address potential information leakage in LLM serving infrastructures. In this paper, we report novel attack strategies designed to exploit such timing side channels inherent in LLM deployments, specifically targeting the Key-Value (KV) cache and semantic cache widely used to enhance LLM inference performance. Our approach leverages timing measurements and classification models to detect cache hits, allowing an adversary to infer private prompts with high accuracy. We also propose a token-by-token search algorithm to efficiently recover shared prompt prefixes in the caches, showing the feasibility of stealing system prompts and those produced by peer users. Our experimental studies on black-box testing of popular online LLM services demonstrate that such privacy risks are completely realistic, with significant consequences. Our findings underscore the need for robust mitigation to protect LLM systems against such emerging threats.

Index Terms—LLM, KV cache, semantic cache, side channels.

Received 27 April 2025; revised 9 September 2025; accepted 13 October 2025. Date of publication 17 October 2025; date of current version 28 October 2025. The work of Linke Song, Wenhao Wang, Wei Song, Dan Meng, and Rui Hou was supported in part by the National Natural Science Foundation of China under Grant 62272452, in part by the Strategic Priority Research Program of Chinese Academy of Sciences under Grant XDB0690100, and in part by the Research Grant from Huawei. The associate editor coordinating the review of this article and approving it for publication was Prof. Guowen Xu. (Linke Song and Zixuan Pang contributed equally to this work.) (Corresponding author: Wenhao Wang.)

Linke Song, Wenhao Wang, Wei Song, Dan Meng, and Rui Hou are with the State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100045, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: wangwenhao@iie.ac.cn).

Zixuan Pang and Yier Jin are with the University of Science and Technology of China, Hefei 230052, China.

Zihao Wang and XiaoFeng Wang are with Nanyang Technological University, Singapore 639798.

Hongbo Chen is with Indiana University Bloomington, Bloomington, IN 47405 USA.

Digital Object Identifier 10.1109/TIFS.2025.3622954

I. INTRODUCTION

LARGE Language Models (LLMs) are widely used in applications such as chatbots [1], search engines [2], and coding assistants [3]. However, LLM inference is resource-intensive, requiring substantial computational power and memory due to the model’s vast parameters, numerous layers, and large context sizes. Improving LLM inference performance has thus become essential, leading to solutions such as weight quantization [4], [5], [6], model compression [7], and algorithm optimization [8], [9]. While these approaches reduce latency and improve inference efficiency, their privacy implications remain less clear.

In this paper, we conduct the first security analysis of performance optimization techniques employed by modern LLM systems that serve multiple users or applications concurrently. Our research reveals significant information leaks arising from distinct side channels introduced by these techniques. Specifically, current LLM performance optimizations use shared caches to reduce computation and storage overhead during inference. However, memory sharing, cache contention and eviction and task scheduling among different users and applications can interfere with user requests, creating noticeable timing side channels. Exploiting these side channels can expose private prompts from other users or applications.

A. LLM Cache Channels

We examined various caches in LLM systems, which not only reduce the computational cost of LLM inference but also improve user experience by lowering service latency. We found that these caches can be misused to infer proprietary system prompts or sensitive prompts from peer users. These prompts may contain private user information and also hold commercial value, as they enable an LLM to carry out various downstream tasks without additional fine-tuning. We identified two primary cache channels:

- *Leakage from the KV cache.* For each inference request, the LLM maintains an in-memory state called the KV cache, which is reused throughout the request’s entire service time. Due to the causal attention mask in LLMs, each token’s activations are influenced only by preceding tokens in the sequence. Thus, if multiple requests share a common prefix, the key and value embeddings for those prefix tokens are identical across sequences. To optimize the KV cache’s memory usage, the system identifies matching prompt prefixes across multiple

requests and shares their key and value embeddings in memory at runtime [9], [10]. This sharing occurs when prompts include a common prefix, which frequently happens with few-shot examples, chatbot system prompts, or prompt templates [11]. For example, it has been noted that Claude’s prompt caching feature can reduce costs by up to 90% and decrease latency by up to 85% for long prompts [12].

- *Leakage from the semantic cache.* The semantic cache boosts LLM performance by caching responses based on the semantic content of the requests. For example, for the prompts “give me suggestions for a comedy movie” and “recommend a comedy movie”, the LLM system can detect their semantic similarity and return similar responses without querying the LLM backend. Experiments show that when GPTCache is integrated with OpenAI’s service, response speed can be improved by a factor of 2 to 10 upon a cache hit [13].

B. Challenges and Solutions

A straightforward way to exploit these vulnerable caches is to directly search the prompt space for one that triggers a cache hit. However, this method faces multiple hurdles. First, the time difference resulting from hitting a single cache block is often minimal and can blend with GPU system noise and fluctuations in voltage and power, making it difficult to detect and exploit. Second, the KV cache only works when prompts share a common prefix, limiting attack opportunities. Additionally, the vastness of the prompt space makes it infeasible to systematically test every potential prompt to find a cached one. Complicating matters further, the attacker’s own requests might be cached during the process, introducing additional noise and potentially causing the victim’s cached data to be evicted.

To address these challenges, we developed various attack strategies to exploit LLM side channels. Specifically, we use a threshold-based classification model to detect token-level KV cache hits based on offline timing measurements. We observe that online detection accuracy can be substantially improved with only a few repeated trials. To reduce the search space for the KV cache channel, we propose an incremental search algorithm that capitalizes on the requirement for prompts to share a common prefix, allowing us to recover the victim’s prompt token by token. For the semantic cache channel, we design an algorithm to select the most representative prompts as the attacker’s requests, given a targeted semantic focus. To minimize interference from the attacker’s own requests, we introduce a mechanism to clear cached data via batches of irrelevant requests. Our method also ensures the attacker’s requests remain distinct by computing their semantic similarities.

C. Experimental Studies

In our study, we confirmed timing leakages in open-source projects, including SGLang [10], Langchain [14], and GPTCache [13]. We further demonstrate the feasibility of deducing proprietary system prompts (i.e., *prompt stealing attack*) and inferring sensitive requests from neighboring users (i.e., *peeping neighbor attack*).

For the prompt stealing attack, our evaluation indicates that the accuracy of detecting per-token cache hits or misses in the KV cache is 99%, with a false positive rate (FPR) of 0.003. Using the incremental search algorithm, we recovered the system prompt token by token, requiring an average of 112.72 queries per recovered token. This approach achieved an average recovery accuracy of 89.0% and a corresponding FPR of 0.04. For the peeping neighbor attack, our measurements show an 81.4% accuracy in distinguishing hits from misses, with an average FPR of 0.045 in a single trial. This accuracy improved to 95.4% with a 0.056 FPR after 5 trials under GPTCache’s default settings. We further observed that it is possible to infer the documents processed by a victim user in a vulnerable LLM application, even when using standard commodity LLM services. Moreover, our black-box study of existing online services shows that popular LLM systems—such as Claude, DeepSeek, and Azure OpenAI—employ KV or semantic cache sharing to cut costs, rendering them susceptible to timing side-channel attacks.

Finally, we propose initial defenses against these side-channel risks. To mitigate KV cache leakage, we recommend sharing prefix caches only in batches of at least k tokens ($k \geq 2$). Although this increases the prompt search space and thus the required number of guesses, the larger timing differences for sharing multiple tokens also make classifiers more robust. Consequently, attacks remain accurate but incur higher query overhead. To address semantic cache leakage, we advise anonymizing privacy-related content in user inputs before performing semantic-similarity searches. Our experiments show that this method adds modest overhead (around 4%).

D. Contributions

Our paper makes the following contributions:

- *New discovery.* We identified new timing side channels in both open-source and online LLM serving systems, arising from the sharing of KV caches and semantic caches.
- *Novel exploit strategies.* We introduced new attack strategies to leverage the inherent side channels in LLM inference optimizations, enabling two distinctive attacks: prompt stealing attack and peeping neighbor attack.
- *Experimental validations, real-world measurements and mitigations.* We validated the side-channel leakages locally on prominent LLM systems and conducted a black-box measurement study of popular online LLM services. We also presented preliminary mitigation measures for these risks.

E. Responsible Disclosure

We disclosed our findings to all relevant developers (SGLang, GPTCache, etc.) and LLM service providers (OpenAI, Claude, etc.) upon identifying the side channels in September 2024. At the time of this manuscript’s preparation, we received positive responses from the SGLang team, which noted that we were among the first two groups to report this issue, both within the same week. Moreover, we were the first to raise the topic during the SGLang development meeting, and we are now working closely with their team on a resolution.

F. Comparison with concurrent and follow-up works (Table. 1)

Concurrently and independently to our research, Wu et al. proposed PROMPTLEAK [15], an attack that exploits the Longest Prefix Match (LPM) scheduling policy in SGLang, which prioritizes requests with longer prefix matches, to leak user prompts. Their method performs collision attacks by sending carefully crafted batched prompts. When a request shares more KV cache entries than others, SGLang prioritizes it, enabling adversaries to extract victim tokens. In contrast, our work identifies a more general KV cache side channel that does not depend on LPM-based scheduling and thus remains effective even without prefix-based prioritization. Moreover, we are the first to reveal a semantic cache side channel, propose practical mitigations, and conduct real-world measurements to assess leakage risks in commercial LLMs.

Inspired by our findings, Gu et al. conducted a subsequent large-scale measurement study on prompt caching in real-world LLM services [17], identifying its presence in 8 out of 17 evaluated providers. More recently, Zheng et al. investigated timing side channels in LLMs through the InputSnatch attack [16]. However, their work lacks the optimized search strategy we introduce for efficient request recovery and suffers from practical limitations. Specifically, their KV cache attacks on vLLM can extract blocks of 16 tokens but are limited to template-dependent scenarios, requiring attackers to have prior knowledge of fixed input structures. Additionally, their semantic cache attacks on GPTCache aim to infer patterns from retrieved documents but do not address self-induced interference from the attackers' own queries and lack eviction strategies to mitigate adversarial noise. In contrast, our approach enables template-free extraction, independent of application-specific formats, and incorporates eviction-controlled probing to systematically eliminate interference. Furthermore, we validate the effectiveness of our method in a realistic setting: a document summarization service powered by a commercial LLM API.

In general, PROMPTLEAK and InputSnatch achieve higher prompt recovery rates and lower per-token recovery costs than our attack, but at the expense of much stronger assumptions. Specifically, PROMPTLEAK depends on the LPM scheduling policy in SGLang, while InputSnatch relies on predefined template structures and fixed sentence blocks known to the attacker. By contrast, our method only assumes cache sharing and does not require such constraints. Even under this weaker and more realistic model, our attack achieves a reasonable single-token recovery accuracy and can recover a meaningful number of tokens — 81 tokens with only 519 guesses (Section IV-A). In practice, attackers could recover even more tokens by employing a stronger next-token predictor and increasing the maximum number of attack queries.

Availability. All the code and datasets necessary to reproduce our experiments are publicly available at: <https://github.com/Maxppddcsz/llm-sidechannel>. The demos for our attacks are available at: <https://sites.google.com/view/early-bird-catches-the-leak/>.

II. BACKGROUND

A. LLM Serving Systems

In this paper, we explore shared LLM deployments serving multiple users or applications within computing systems. This setup is common in commercial public services (e.g., ChatGPT) and local LLMs tailored to specific tasks. In these scenarios, LLMs optimize latency and throughput via memory usage optimization, efficient batching, and GPU scheduling. These factors can introduce interference among concurrent requests, potentially leading to observable timing side channels.

1) *LLM Serving Modes:* The LLM service offers two operation modes. In non-streaming mode, the response is fully generated and then delivered once the request has been processed. However, this approach can result in an extended waiting period for long completions. To achieve faster responses, the streaming mode is available. In this mode, the LLM emits tokens sequentially, allowing users to view the beginning of the completion while the remaining tokens are still being generated. Streaming is the preferred method for interacting with LLMs, especially in chatbot scenarios where real-time conversation is essential. Popular LLM applications (e.g., ChatGPT) use a system prompt containing task definitions, examples, and safety rules to guide their behavior. This prompt is typically static and shared among all users.

2) *Metrics:* Latency measures how long it takes for an LLM to respond to a user's query. Low latency is particularly important for real-time interactions, such as chatbots. Time to First Token (TTFT) is the interval from the moment a user submits a prompt until receiving the first token of the response. It reflects the initial processing delay and serves as a crucial indicator of user-perceived responsiveness. This paper examines the risks arising from optimizing an LLM's serving latency and employs TTFT as the primary metric for side-channel observations.

B. Serving Backend

Most LLMs rely on the Transformer architecture, which uses *Attention* [18] to pinpoint the most relevant parts of the input. Core to this mechanism are Query (Q), Key (K), and Value (V) embeddings: Q specifies what to seek at each position, K determines how to match relevant information across the sequence, and V provides the data retrieved upon a match. LLM inference consists of two stages: the *prefill phase* and the *decoding phase*. In the prefill phase, the LLM processes the request prompt by converting it into a sequence of tokens, each mapped to a numerical representation called embedding. It then computes the K and V embeddings for each token across all attention layers, enabling the generation of *the first token* in a single step. In the decoding phase, the LLM generates each subsequent token by using the prefilled context and the previously generated token. For each layer, it computes the Q, K and V embeddings for the new token and attends to all existing context tokens. Unlike the prefill phase, decoding processes only one token at a time.

1) *Memory Management of KV Cache:* The attention mechanism in LLMs requires computing pairwise similarities

TABLE I
COMPARISONS WITH CLOSELY-RELATED WORKS

	Independent of scheduling policy?	Independent of prompt templates?	Cache eviction supported?	Real-world measurement?	KV cache channel?	Semantic cache channel?	Mitigations
PROMPTLEAK [15]	✗	✓	✓	✗	✓	✗	✗
InputSnatch [16]	✓	✗	✗	✗	✓	✓	✗
This work	✓	✓	✓	✓	✓	✓	✓

TABLE II
EXAMPLES OF USER PROMPTS THAT CONTAIN PRIVATE ATTRIBUTES

Use cases	Prompts
Healthcare	Compose a meeting agenda for an interdisciplinary team discussing the treatment plan for [Name] with [medical condition].
Travel planning	I'm [flying/driving] to [destination] with [Name] for a leisurely trip, and we'll be staying at [hotel] for [number of days]. Can you create a comprehensive packing list organized by category?
Business planning	Act as an expert business plan writer, and help me generate a product and services section of my [business type] business plan. My company is [business] called [business name] that specializes in [USP or specialization].
Performance review	I'm preparing for the annual performance review of an employee named [Name]. [Name]'s role involves [roles]. Draft a performance review for [Name] and suggesting improvements in [area of improvement].
E-mails	Draft an e-mail to the [company] on [subject].
Cover letter	Write a conversational cover letter for [Name] for a job application as a [position] at [company].
Out-of-office message	Write a short out-of-office message. Reason: [vacation]. Dates: [month and dates]. Person to contact in an emergency or for immediate help: [name] at [email address].

among tokens, which leads to quadratic complexity with respect to sequence length. To address this, KV caching stores the K and V embeddings in GPU memory, avoiding redundant computations and reducing the computation cost to scale linearly with sequence length. Early LLM systems statically allocated large memory regions for KV caches to accommodate unpredictable output lengths, leading to significant internal and external fragmentation. vLLM mitigated this with PagedAttention, which divides the KV cache into blocks accessed via a lookup table [9]. This enables efficient memory sharing across requests. Modern frameworks like Nvidia's TensorRT-LLM and Huggingface's TGI adopt similar techniques, but the security implications of KV cache sharing remain underexplored, leaving a critical gap in existing research.

C. Threat Model and Assumptions

1) *Threat Model*: In this paper, we examine the security risks of deploying a shared LLM to serve multiple users or applications within a single computing system. Specifically, we consider two main scenarios. First, an LLM service provider offers public APIs that registered users can employ to send requests, all of which are processed by the same underlying serving system. A victim user may establish a proprietary system prompt to power a popular LLM application, while an attacker could leverage the same LLM APIs to infer this system prompt, thereby gaining financial benefits or bypassing embedded safety instructions. Second, public LLM applications—such as chatbots (e.g., GPT-4) or document analysis services (e.g., AnythingLLM [19], etc.)—handle concurrent user requests via the same LLM serving system. If the application itself relies on a public LLM API, these requests are generally routed through the same developer's API key. An attacker could register as a user of the application to detect whether specific requests have been submitted by

others and infer, for instance, users' interest in certain topics or uploaded files. They could also monitor requests over time to detect private attributes or sensitive personally identifiable information (PII) (Table II). In both scenarios, the attacker's and the victim's requests share the same platform and thus same cache resources. This shared environment can produce interference and create observable timing side channels—the core subject of our investigation. The attacker needs only black-box access to the underlying model, without knowledge of its architecture or weight parameters. However, the attacker must first examine the system's leakage profile in an offline phase to understand how different inputs affect timing, enabling them to craft queries that exploit the cache-induced timing discrepancies.

In this paper, we explore the side-channel risks in both local and remote LLM services. For local settings, we assume a stable network connection between client and server. For remote services, previous works such as NetCAT [20] and NetSpectre [21] have addressed mitigating noise caused by unstable connections and jitters, particularly in CPU cache side-channel attacks. Extending such noise-reduction strategies to remote LLM scenarios remains an avenue for future research. We do not consider hardware side channels tied to GPU micro-architectures [22], [23], [24]. Instead, our focus lies on software caches maintained by the LLM serving system, making our attacks applicable across various hardware platforms (CPUs, GPUs, ASICs, etc.).

2) *Assumptions*: For end-to-end attacks, we assume that the adversary has certain prior knowledge of the target user or system prompt. Specifically:

- **Prompt Stealing Attacks** (subsection IV-A): The attacker is assumed to have access to a public dataset of system prompts that can be used to fine-tune the next-token predictor. Such datasets are semantically diverse and representative of real-world prompt engineering practices. In

our evaluation, the victim randomly selects a small subset (e.g., 200 prompts) as its proprietary system prompt, while the attacker uses the remaining prompts as training data. This reflects realistic scenarios where production systems employ a limited number of carefully crafted prompts, while public repositories offer abundant similar data for adversarial training.

- **Peeping Neighbor Attacks** (subsection IV-B): We assume the attacker is focused on a particular topic (e.g., healthcare, travel planning) and has a public set of prompt templates containing sensitive private attributes such as personal names, medical conditions, travel destinations, or business details. These templates can be obtained from public datasets or generated using paraphrasing tools. The attacker uses them to probe semantic caches, where timing differences between cache hits and misses may reveal whether the victim's request contains the targeted attributes.
- **Document Inference Attacks** (subsection IV-C): Because KV-cache sharing typically requires requests to share a common prefix, we assume the attacker knows either the full document or at least its prefix (i.e., the initial portion of the content). Even if the document's content is known, detecting its submission to an LLM service may disclose an organization's interests, posing serious privacy and competitive risks across different sectors.

III. ATTACKS

A. Overview

Creating effective prompts is a challenging task that requires substantial effort, particularly in scenarios like in-context learning where extensive data is needed to optimize LLM performance. Furthermore, prompts can include personal or sensitive information, making them valuable assets that must be safeguarded. For instance, Samsung Electronics has prohibited employees from using Chatgpt to prevent accidental disclosure of confidential data to OpenAI [25].

In our research, we investigated two types of attacks. The first is the *prompt stealing attack (PSA)*, which targets system prompts. A system prompt defines the model's operational behavior and may incorporate carefully crafted business logic, private data, or safety-related instructions. Consequently, LLM application developers treat it as confidential intellectual property [26]. Moreover, once exposed, the system prompt could facilitate other attacks, such as jailbreaking. The second is the *peeping neighbor attack (PNA)*, which focuses on uncovering the semantics of another user's prompt. Since these prompts may contain personally identifiable information (PII) or other sensitive data, any disclosure poses a substantial risk to user privacy. There are three entities involved in these attacks: the server (S), the victim user (C), and the attacker (A). The attacker's goal is to infer the prompt submitted by the victim user. The attack proceeds in two phases. In the *offline phase*, the attacker studies how a request alters the server's state and how these modifications manifest in the latency of subsequent requests. Critically, these timing profiles stem

primarily from the system's optimization techniques rather than from a specific model or parameter set.

In the *online phase*, the attacker leverages insights gained during the offline phase to craft requests that exploit the identified timing properties. Initially, S is in state $State_0$. When C issues a request, the state changes to $State_1$, reflecting updates like modifications to the KV or semantic cache. These state transitions can affect the performance of later requests. To track the system state, A regularly sends a request r at intervals starting from time t_{start} , measuring the resulting latency $l = t_{end} - t_{start}$, where t_{end} denotes the time point when the first token in the response arrives. By analyzing these latency readings, A can infer the prompt submitted by C .

B. Prompt Stealing Attacks (PSA)

1) *Background*: KV caching is a widely adopted optimization in LLM which avoids redundant computation during autoregressive generation by storing key and value embeddings. Representative implementations include PagedAttention in vLLM [9], which detects shared prefix segments at runtime for KV cache reuse, and RadixAttention in SGLang [10], which efficiently manages shared KV caches in prefix tree.

2) *The side channel*: Sharing KV caches can introduce a timing side channel. In the prefill phase, if a request's prefix matches one already stored in the KV cache, it will be processed more quickly. Since most LLMs stream their outputs (i.e., token by token), it is possible to measure the fine-grained TTFT and detect timing discrepancies associated with cache hits versus misses. Consider a model with 7 billion parameters (e.g., Llama-7B) running on an A100 GPU with 312 TFLOPS of computational power and a memory bandwidth of 1.5 TB/s. In the best case, with full GPU utilization, a cache miss for a single token during the prefill phase may take:

$$\begin{aligned} \text{prefill time (miss)} &= \#tokens \times \frac{\#parameters}{\text{GPU compute bandwidth}} \\ &= \frac{1 \times (2 \times 7B) \text{ FLOP/token}}{312 \text{ TFLOP/s}} \\ &\approx 0.045 \text{ ms.} \end{aligned}$$

By contrast, if the token hits the cache, the time is dominated by loading the precomputed KV cache from HBM (assuming 16-bit precision parameters):

$$\begin{aligned} \text{prefill time (hit)} &= \#tokens \times \frac{\text{KV cache per token}}{\text{GPU memory bandwidth}} \\ &= \frac{1 \times (2 \times 4096 \times 32) \times 2 \text{ Bytes/token}}{1.5 \text{ TB/s}} \\ &\approx 0.35 \mu\text{s.} \end{aligned}$$

These gaps become larger for more complex models or when serving multiple requests concurrently. For instance, on a Llama-3.1-70B-Instruct-GPTQ-INT4 model (70 billion parameters at 4-bit precision), the prefill time for a token miss is about 0.45 ms, while a hit is roughly 0.22 μs .

These timing differences underpin the prompt stealing attack, which leverages KV cache sharing with the same prefix. After sending a request to the LLM, an attacker can observe

TTFT to infer prefix matches. Building on this observation, we devised an incremental search algorithm to recover prompts on a token-by-token basis in subsection IV-A.

C. Peeping Neighbor Attacks (PNA)

1) *Background*: Semantic caching (e.g., GPTCache [13]) stores prior requests and their corresponding responses. Upon receiving a new request, it measures semantic similarity with cached requests. If the similarity surpasses a certain threshold, the system returns the cached response; otherwise, it queries the LLM again. This technique reduces costs, improves performance, and is widely adopted in frameworks like LangChain [14] and LlamaIndex [27].

2) *The side channel*: Unlike KV caching, which reuses data only for identical prompt prefixes, semantic caching allows reuse based on semantic similarity beyond a predefined threshold. However, sharing a semantic cache among multiple users can inadvertently reveal their requests. Cache hits provide responses in mere milliseconds, whereas cache misses can take several seconds—creating exploitable timing differences. This discrepancy enables the attacker to infer the semantics of concurrent requests issued by nearby users, a scenario we refer to as the *peeping neighbor attack*. Despite this, when the attacker tries to match a victim's request semantically, the attacker's own requests may also be cached, introducing noise into subsequent attempts. To address this challenge, we propose an efficient search algorithm that both minimizes the caching effects of the attacker's own requests and improves the detection rate for the victim's request, detailed in subsection IV-B.

IV. SIDE-CHANNEL ANALYSIS AND EVALUATION

In this section, we present our empirical analysis of the identified side channels and describe strategies for their efficient exploitation. All experiments were conducted on a Lenovo server equipped with two Intel Xeon E5-2678 v3 CPUs (12 cores at 2.50 GHz each), 100 GB DDR4 memory, and two NVIDIA A100 PCIe GPUs (80 GB memory each). The system ran Ubuntu 22.04 (kernel 5.15.0-125-generic) with GPU driver 550.127.05, CUDA 12.4, and PyTorch 2.4.0. We used open-source models from the Llama family as the underlying LLM, adhering to their default hardware and software configurations for all evaluations.

A. Analysis on PSA

1) *Attack Setup*: With increasing concerns about client data leakage in public LLM services, local enterprise LLMs have become increasingly popular [28]. In our study, we focus on a use case where the LLM API service is constructed from open-source projects within a *local network environment*. As described in subsection II-C, we consider a scenario in which a victim develops a popular LLM application (e.g., a chatbot) using a proprietary system prompt via the LLM service. The attacker interacts with this LLM application over the local network, measures the TTFT, and attempts to uncover the system prompt based on timing discrepancies. Specifically, the LLM service uses the SGLang backend API

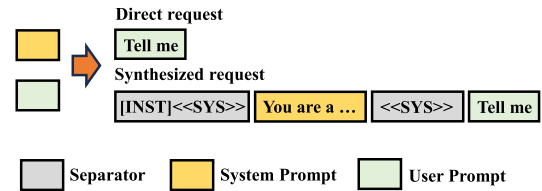


Fig. 1. LLM API servers like OpenAI allow user input through both direct requests (top) and synthesized requests via a template (bottom).

server [10], which supports KV cache sharing for common prefixes. Notably, LLM API servers including OpenAI allow users to define various roles, such as “system” and “user”, within their requests [29], or send requests directly. Based on this capability, we categorize requests into two modes: synthesized and direct. The victim's LLM chatbot, built on the FastChat framework [30], explicitly supports both, as shown in Figure 1. In direct mode, the user sends requests directly to the SGLang backend, whereas in synthesized mode, the full prompt is created by concatenating messages from each role according to predefined templates.

We consider that the victim's chatbot employs a proprietary system prompt for the “system” role in the synthesized mode, while the user's inputs fall under the “user” role. In the synthesized mode, this system prompt is prepended at every conversational turn to form the complete prompt sent to the SGLang backend. Notably, BloombergGPT [31] serves as a real-world example of a purposely built LLM for financial use, deployed for internal use at Bloomberg. It leverages 3-shot prompting to handle domain-specific tasks more effectively, safeguarding sensitive data and maintaining control over proprietary financial processes. As illustrated in Figure 2, an attacker can masquerade as a chatbot user, submitting either a direct request or a synthesized request that includes the static system prompt. When the LLM processes these synthesized prompts, it retains the separator and system prompt in the KV cache used by the SGLang backend, expediting subsequent requests that share partial prefixes of the system prompt. In this attack, the attacker first submits a synthesized request to cache the system prompt, then employs direct queries to reveal it through timing leakages.

2) *Characterizing the Leakage*: We began by examining the timing difference between a cache hit and a miss for a single token across different model sizes. Specifically, we tested SGLang v0.3.0, which uses a radix tree to manage KV cache blocks for efficient prefix matching. Two models were evaluated: Llama-3.1-8B-Instruct and Llama-3.1-70B-Instruct-GPTQ-INT4. System prompts were derived from the gabrielchua/system-prompt-leakage [32] dataset. Using 15-token prompts differing only in the last token—ensuring a shared-prefix difference of exactly one—we collected TTFT measurements over 4,000 runs. Figure 3 illustrates the resulting time distributions, revealing a pronounced distinction between hit and miss scenarios for both models.

Based on these observations, a simple classifier can be built to categorize the last token in the prompt as a hit if its latency is below a predefined threshold. However, as prompts

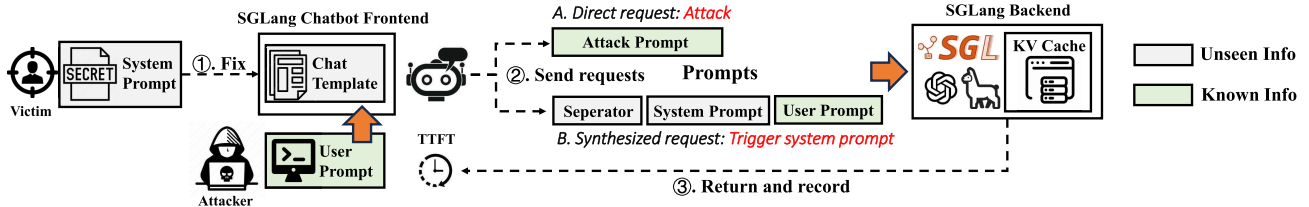


Fig. 2. Overview of prompt stealing attacks.

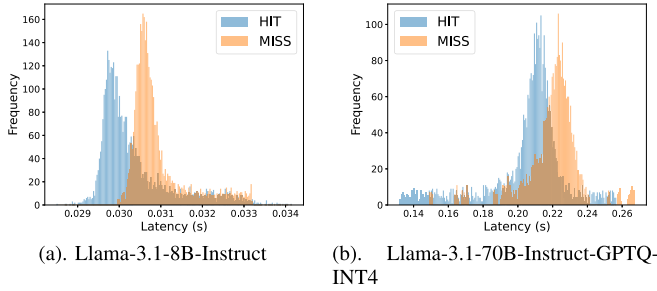


Fig. 3. Latency distribution of one token hit and miss.

grow longer, the latency also tends to rise due to increased computational demands, necessitating different thresholds for each prompt length. To address this, instead of the absolute latency, we measured the relative latency difference between hit and miss cases for last tokens across prompts of varying lengths (1–200 tokens). Our findings reveal that even though the absolute latency varies, this difference remains stable across prompt length, and significantly larger than the variance within miss cases, enabling a single unified threshold for reliable hit detection.

In real-world evaluations of our classifier, we noted that TTFT varies due to factors such as GPU system noise and power fluctuations, weakening the effectiveness of a fixed classification threshold, as shown in Figure 3. To mitigate this, we first construct a “miss prompt” by appending a rare token to the partially recovered prompt (i.e., requests without predicted tokens) and designate the prompt being evaluated as hit or miss as the “candidate prompt”. Then, within a brief time window, we simultaneously collect TTFT data for both the miss and candidate prompts. By using the difference between these measurements rather than absolute values, we establish a threshold-based classification that remains robust against temporal system variations.

To illustrate the classifier’s effectiveness, consider an example using the Llama-3.1-70B-Instruct-GPTQ-INT4 model under our evaluation settings. We built a classifier to determine whether the last token of prompts hit the KV cache. We tested the classifier using 4,000 hits and 4,000 misses drawn from randomly selected system prompts, achieving a TPR of 0.88 and an FPR of 0.10. To further mitigate noise, we employ multi-sampling by collecting n TTFT samples per token. A token is classified as a hit only if the number of detections exceeds a threshold k . This leverages binomial statistics to amplify the TPR while suppressing the FPR. With $n = 10$ and $k = 5$, the final classifier attains a TPR of 0.99 and an FPR of 0.003.

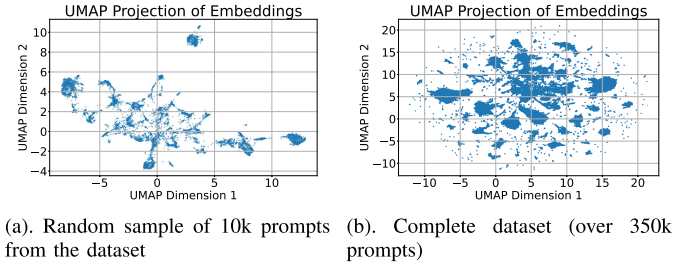


Fig. 4. UMAP projection demonstrating the dataset’s heterogeneity: As sampling density increases, the visualization reveals expanded dimensional ranges with distinct semantic clusters and isolated points, highlighting the complex multidimensional structure of the prompt dataset.

3) *End-to-end Attacks*: We built a local chatbot using the FastChat as the victim application. Its backend API server is configured with SGLang v0.3.0, and the model deployed by victim is Llama-3.1-70B-Instruct-GPTQ-INT4. For evaluation, we used the gabrielchua/system-prompt-leakage dataset, containing over 350,000 synthetic system prompts. We applied Llama-3.1-8B-Instruct to compute embeddings for each prompt and visualized the results using Uniform Manifold Approximation and Projection (UMAP), which reduces the dimensions of datasets and enables effective visualization of data distribution [33]. UMAP visualization confirmed substantial semantic diversity across this dataset, reflecting the varied topics and styles found in real-world prompt engineering. As shown in Figure 4, this semantic heterogeneity increases as the dataset expands, significantly complicating predictive modeling. Consistent with the threat model detailed in subsection II-C, we assume the attacker has a public dataset of system prompts that mirrors those of the victim. To reflect real-world conditions—where production systems typically implement a small number of carefully crafted system prompts—we randomly select 200 prompts as the victim’s configuration while reserving the remainder as the attacker’s training corpus. This experimental design deliberately simulates the “needle in a haystack” challenge faced in realistic attack scenarios, where adversaries must identify specific target prompts within a substantially larger heterogeneous corpus.

To streamline the search process, as illustrated in Figure 6, we propose an incremental token-by-token approach to recover the target prompt. This approach relies on multiple components: a *classifier* for validation, a *next-token predictor* to estimate token probabilities, and a *sampler* that selects candidate tokens based on the predictor’s temperature setting. The *next-token predictor* is fine-tuned on the public system prompt


```

def get_ttft(text):
    start_time = time.perf_counter()
    response = requests.post(
        {..., max_tokens=1,...},
        stream = True
    )
    for line in response.iter_lines():
        if line:
            end_time = time.perf_counter()
            break
    return end_time - start_time

def complete(text):
    response = requests.post(...)
    for line in response.iter_lines():
        if line:
            data = json.loads(line)
            if data.get("end_of_sequence",
                False):
                break

complete(triggering_prompt)
time.sleep(0.2)
ttft = get_ttft(predicted_prompt)

```

Fig. 5. Code for measuring response latency in PSA.

dataset available to the attacker, allowing it to forecast the next token given the already retrieved tokens. For each candidate token at position i , we feed the partially reconstructed prompt and miss prompt into the LLM to obtain TTFT difference (same method in subsection IV-A), which then serves as the input to the *classifier*. If the classifier identifies this token as a cache hit, the token is appended to the prompt; otherwise, a *repetition penalty* is applied by adjusting the probability distribution of the current token (in our implementation, this penalty is applied by halving the sampling probability for an incorrect token in the next round).

In constructing the *next-token predictor*, we adopt Llama-3.1-8B-Instruct as the base model, chosen as it shares an identical tokenizer with the victim LLM. We fine-tune this model using the attacker's training dataset. Given the partially recovered prompt and the chosen temperature, the predictor generates a probability distribution for the next token, while temperature scaling introduces variability into the prediction. Our predictor is not heavily optimized; it was fine-tuned on a single NVIDIA A100 PCIE GPU for only a few hours with limited training resources. With larger models, more epochs, and bigger batch sizes, the predictor's performance could be further improved.

To obtain TTFT values, the attacker first clears both the victim's and the attacker's requests from the cache. Next, the attacker measures the TTFT for their own request after the victim's system prompt has been cached. Below, we describe how the TTFT is gathered and how caches are flushed.

- *Timing measurement.* As shown in Figure 5, the attacker begins by issuing a synthesized request containing the targeted system prompt. Once the end-of-sequence token is received in the POST response, a short delay is introduced, ensuring the system prompt resides in the KV cache. The attacker then

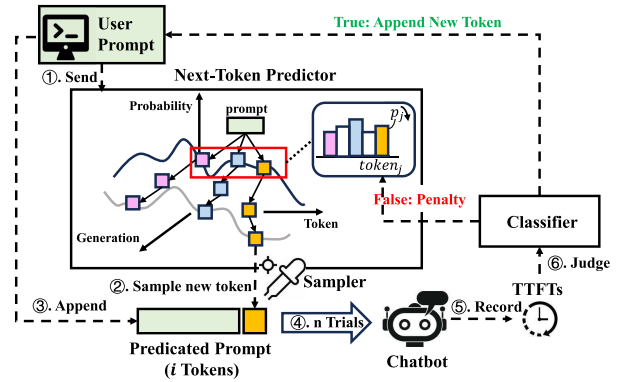


Fig. 6. Efficient token-by-token request recovery.

sends a direct request via a POST call using the predicted prompt, configured to generate only 1 output token. The TTFT is computed as the interval between sending the request and detecting the first non-blank token in the streamed response.

- *Flushing the caches through eviction.* We observed SGLang provides a *flush_cache* API [34] that efficiently clears the cache. However, for our end-to-end attack scenario, we chose not to use this API, as it is unlikely to be accessible to attackers in real-world environments. Instead, we employed a more robust method of evicting the KV cache by issuing batches of irrelevant requests. These irrelevant requests consist of diverse texts with rare prefixes to rapidly fill the GPU memory and trigger cache eviction. Under default SGLang settings, sending 15 such requests (each containing about 200 tokens) was sufficient to trigger eviction in about 5 seconds. This approach proved successful in 100% of our 10,000 tests.

We evaluated both the token recovery accuracy and the average number of queries required per token. The results indicate a success rate of 89.0%, an FPR of 0.04, and an average of 5.64 guesses with 112.72 attack queries needed per recovered token when $n = 10$. The corresponding cost per token recovery is approximately \$0.16 using OpenAI o1, \$0.012 with OpenAI o3-mini, and \$0.001 with Deepseek-Chat (as of March 5, 2025). Out of 200 victim prompts, we successfully recovered an average of 11.32 tokens for the top 100 prompts, and 17.84 tokens on average for the top 50 prompts. The maximum number of tokens recovered for a single prompt was 81, achieved with just 519 total guesses. Notably, we limited the token prediction attempts to a maximum of 80 per position, ceasing further attempts if the correct token was not identified within these trials. Consequently, the primary constraint in reconstructing entire system prompts stems from the precision of the predictor rather than the accuracy of the classifier—an aspect that falls outside the scope of this study. In real-world attacks, the attacker can recover additional tokens by deploying a more advanced next-token predictor and increasing the maximum number of attack queries. A demo for the attack is presented on our website [35].

4) *Discussions:* Currently, we assume that the attacker has access to a public dataset of system prompts that can be used to fine-tune the next-token predictor. When a generic (i.e., untuned) predictor is employed, it tends to generate more

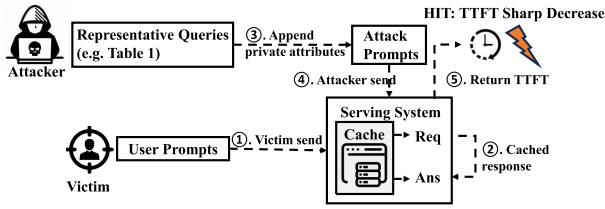


Fig. 7. Peeping neighbor attacks.

false predictions, which in turn requires additional verification queries to distinguish genuine cache hits from prediction errors (i.e., to reduce false positives). This added overhead increases the effort and resources needed for an attacker to recover tokens. However, these challenges raise the cost of the attack rather than eliminating the underlying vulnerability. The core timing side channels we identified in KV-cache mechanisms remain exploitable regardless of predictor quality. With sufficient resources and persistence, attackers can still leverage these timing differences to recover tokens, albeit at a higher cost. To evaluate how predictor quality affects attack efficiency, we conducted the attack using the untuned Llama-3.1-8B-Instruct model as the generic predictor. Our results show a recovery rate of 69%, with an average of 163.48 guesses and 5,231.30 attack queries per token (here we set $n = 16$ and $k = 8$ to reduce false positives, and limit the token prediction attempts to a maximum of 1,000 per token), and an FPR of 1.5%. Further analysis reveals that, compared with fine-tuned predictors, generic predictors struggle to correctly predict certain rare tokens, leading to a lower recovery rate. Moreover, because generic predictors are less likely to quickly guess the target token, the average number of guesses rises sharply (from 5.64 to 163.48). This further demonstrates that our side-channel-based classifier is robust to false positives, as the attack maintains a low overall false positive rate (1.5%) even under significantly increased guessing attempts.

B. Analysis on PNA

1) Attack Setup: In the PNA attack, we observe that not all user queries contain sensitive data. An attacker typically ignores generic requests like “What’s the weather today?” and focuses on those specific revealing private information. For example, a prompt like “Draft a travel plan for me to Rome in September for a 3-day stay at the Hilton hotel” could expose personal and location details. We assume both the victim and the attacker are normal users of a chatbot service that uses a semantic cache in its backend. By identifying high-risk queries, the attacker can exploit timing side channels to recover private information from other users’ requests. For this purpose, the attacker could compile a list of privacy-related prompts from online sources (Table. II). The attacker has obtained a list of private attributes, and aims to discover connections among these private attributes, e.g., whether a specific user is linked to a particular medical condition.

Figure 7 illustrates the PNA steps. When semantic caching is used, the LLM stores victim requests and serves cached responses for similar queries. To exploit this channel, the

attacker creates requests containing private attributes and monitors TTFT. Since cache hits retrieve responses directly from local storage and bypass LLM inference, they produce a sharp TTFT reduction (from 5 seconds to under 1 second), which far exceeds the timing variations caused by network jitter. Consequently, by monitoring such timing differences, an attacker can infer semantic matches with a victim’s cached request and potentially expose private data.

In practice, users may express the same intent through varied phrasing and may embed diverse private attributes—such as personal names or medical conditions—within their queries. Our objective is to determine how these different attributes affect semantic similarity and whether the resulting variations exceed a threshold described in subsection III-C. If so, such differences can be detected, enabling the attacker to determine whether the victim’s request carries particular attributes, even when the victim rephrases the content.

In our experimental study, we chose LangChain as the LLM serving framework, given its widespread adoption and extensive application ecosystem. The local chatbot system was built using LangChain and integrated with GPTCache for semantic caching. We used gpt-3.5-turbo API as the backend LLM, and MedQuAD [36] as the evaluation dataset. For evaluating semantic similarity, we employed *OnnxModelEvaluation* method in GPTCache, which leverages the albert-duplicate-onnx model.

2) Characterizing the Leakage: Figure 8 outlines our evaluation steps. We illustrate the process with a query template “Compose a meeting agenda...for [Name] with [medical condition]”, denoted as T_0 . Here, both the name and medical condition are treated as private attributes.

Step 1. We randomly sampled 10 names from the Python package *names-dataset* (*Names*) and extracted 10 medical conditions (*Medconds*) from semantically unrelated Q&A pairs in MedQuAD dataset using GPT-3.5-turbo. We then randomly chose 1 name ($Name_0$) and 1 medical condition ($Medcond_0$) as the private attributes to be recovered; the remaining pairs serve as the negative group (described below).

Step 2. Since real-world users may phrase the same content differently, we generated multiple sentences that share the same semantics as T_0 . Specifically, we asked GPT-3.5-turbo to paraphrase T_0 into n variations $\{T_1, \dots, T_n\}$, each filled with $Name_0$ and $Medcond_0$. Following these steps, we created two sample sets: (1) *Positive Group*: Sentences that are semantically similar to T_0 , all containing $Name_0$ and $Medcond_0$. Formally, $\{(T_i, Name_0, Medcond_0) \mid i = 1, \dots, n\}$. (2) *Negative Group*: The original template T_0 populated with other names or medical conditions. Formally, $\{(T_0, Name_i, Medcond_j) \mid i \neq 0 \text{ or } j \neq 0\}$.

Step 3. We split the positive group into two subsets. The first (20% of the samples) is designated as the “positive” reference set, while the remaining 80% forms the evaluation set. We also ensure that the evaluation set’s positive and negative samples are equal in size. Finally, we compute semantic similarities between the evaluation samples and both the positive and negative groups, yielding two respective similarity distributions.

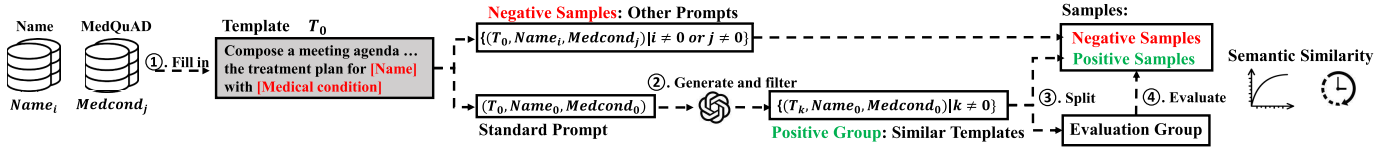
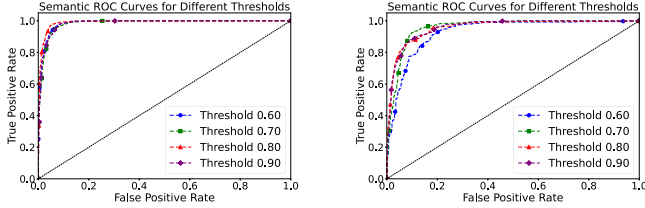


Fig. 8. Evaluating semantic leakage of private attributes.



(a). The “name” and “medical condition” in the negative group are both different from the positive group.

(b). Either the “name” or “medical condition” in the negative group is different from the positive group.

Fig. 9. Leakage profile of semantic cache sharing. We plotted the ROC curve to fingerprint the relationship between the similarity vectors of the positive and negative groups.

We tested similarity thresholds from 0.6 to 0.9 (the default value in GPTCache is 0.8). Figure 9a shows the Receiver Operating Characteristic (ROC) curves for the positive and negative similarity distributions, revealing a clear separation between the two. At the default threshold of 0.8, a TPR of 0.95 can be achieved with an FPR below 0.1. We also examined cases where only one private attribute (either $Name_0$ or $Medcond_0$) matched. Here, the negative group consists of sentences with only one correct private attribute, while the positive group remains the same. Figure 9b shows the semantic distinctions remain substantial: at the default threshold of 0.8, a TPR of 0.85 corresponds to an FPR under 0.1.

3) *End-to-end Attacks*: We consider a typical open-world scenario in which a victim user requests healthcare assistance from an LLM. For instance, the user might submit a query with semantics similar to the template “compose a meeting agenda...” shown in Table II, but with various names and medical conditions. The user may also send queries unrelated to the targeted request. To simulate this, we model the user’s queries as follows:

- **Type-1 (true samples)**: Queries with the specific name (e.g., “Alice”) and specific medical condition (e.g., “heart disease”).
- **Type-2 (false samples)**: Queries that use the same name as the true samples (e.g., “Alice”) but feature different medical conditions, such as “diabetes”, “hypertension”, or “asthma”.
- **Type-3 (false samples)**: Queries with the same medical condition as the true samples (e.g., “heart disease”) but with different names.
- **Type-4 (false samples)**: Unrelated queries.

We assume that the attacker targets the private attribute associations in Type-1 queries. The victim can freely choose different paraphrases while preserving the same underlying semantics. To simulate this, we configure the victim to send

five random requests per round: one Type-1 query (true sample) and four false samples (one Type-2, one Type-3, and two Type-4 requests). To measure the effectiveness of the attack, we use the TPR to assess how successfully the attacker retrieves private attributes from Type-1 requests. We also measure separate FPRs to capture how often the attacker misclassifies each of the three false sample types as positive.

To perform effective end-to-end attacks on the semantic cache, we must eliminate noise from the attacker’s own cached requests. We address this by clearing the cache after each attack round: under GPTCache’s default settings, sending 1,000 semantically unrelated requests suffices to remove any leftover cache entries. These requests were pre-extracted from multi-domain articles. Their semantic irrelevance was validated using SBERT [37], considering pairs with a similarity score below 0.3 as semantically unrelated. In this scenario, we assume an attacker aims to determine whether a particular user (e.g., “Alice”) is associated with a specific medical condition (e.g., “heart disease”). Since the victim may use different phrases, the attacker issues multiple requests to enhance coverage and boost the TPR. However, increasing the total number of requests also raises the risk of false positives, especially if new requests strongly resemble earlier ones. To mitigate this issue, the attacker prioritizes *representative requests* in the request space, thus increasing the overall number of queries while minimizing interference among them.

Representative requests are those that most closely approximate the rest of the request space. To identify them, we use the *distilbert-base-uncased* model by default to generate embeddings and then compute the L2 distance between these embeddings. We then sort the requests by their L2 distances; those with the smallest distances are deemed the most representative and selected as attack requests to maximize coverage. To further expand coverage, we incorporate *orthogonal requests*—requests that are semantically distinct from one another. This reduces the chance that semantic overlaps among the attacker’s own requests degrade accuracy in identifying victim requests. We classify a cache access as a hit if at least one of the attacker’s requests triggers a cache hit in the timing channel. Although this strategy boosts coverage, it also raises the FPR, necessitating a careful balance.

Figure 10 depicts our greedy search algorithm, which iteratively selects the *most representative* candidates (e.g., α) within a semantically similar target space, unless they are overly similar to previously selected ones (e.g., β). This process continues until no suitable candidates remain or when the FPR exceeds a predefined threshold σ .

In the evaluation, each of the 4 victim request types was tested 500 times. In each iteration, a random pair of private

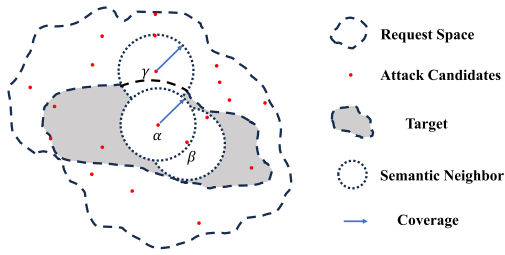


Fig. 10. The greedy search strategy for PNA.

TABLE III
ATTACK ACCURACY FOR THE 4 TYPES OF VICTIM REQUESTS
WITH DIFFERENT NUMBER OF ATTACK TRIALS

#Trials	Type 1 (TPR)	Type 2 (FPR)	Type 3 (FPR)	Type 3 (FPR)
1	0.814	0.116	0.054	0.004
2	0.884	0.142	0.056	0.005
3	0.930	0.146	0.060	0.005
4	0.946	0.150	0.062	0.005
5	0.954	0.152	0.062	0.005

attributes in the Type-1 request was selected. We set $\sigma = 0.06$ and identified 5 *orthogonal* attack requests using the proposed greedy search strategy. Table. III summarizes the TPR for true samples, and the separate FPRs for each false sample type when increasing the attack requests from 1 to 5. Specifically, we recovered 407 victim requests out of the 500 true samples with only 1 attack request, achieving a recovery accuracy of 81.4% with an average FPR of 0.045. With 5 attack requests, 477 victim requests were recovered, demonstrating a recovery accuracy of 95.4% with an average FPR of 0.056. A demo of this attack is available on our website [35].

C. Inferring Documents on Commodity LLM

The KV cache can be shared among the same user, within an organization, or even across organizations [17], creating the potential for cross-user information leaks. In our research, we discovered that such leaks are feasible even in remote attack scenarios, particularly when the target LLM processes documents. To demonstrate this, we utilized a document summarization application powered by a commodity LLM API service, where all user requests are processed using the same developer's API key, and both the victim and the attacker are standard users of the application. We provide an end-to-end demonstration showing how an adversary could infer processed documents from the application through the KV cache side channels. Importantly, such cross-user attacks fundamentally do not require application-specific vulnerabilities, as the KV cache can be shared across organizations [17]. However, our use of the application highlights the privacy risks inherent to LLM-powered applications, even when they fully comply with the guidelines provided by the LLM service.

Note that the observation of the document uploaded to an LLM service, even when the content of the document is known, can expose an organization's interest, with substantial privacy and competitive ramifications across various domains. For example, a law firm relying on an LLM-based document

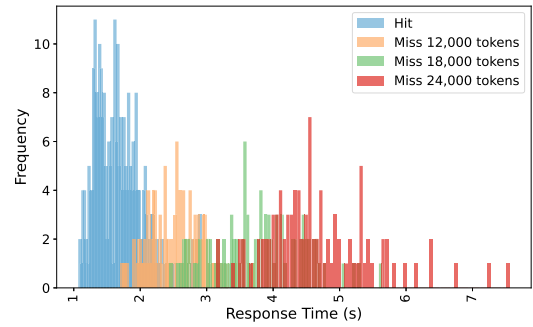


Fig. 11. Timing distribution for hits and misses of processed documents with 12,000, 18,000, and 24,000 tokens.

processor could unknowingly disclose its involvement in complex litigation or pivotal mergers, tipping off opposing parties about strategic decisions before they become public.

1) *The Victim Application*: We implemented the document summarization application with direct summarization (also known as the stuff approach [38]), using the public Deepseek-chat model as the backend LLM API server. The application was built in accordance with Deepseek's guideline and operates by first extracting text from user-uploaded PDF files using the pdfplumber package. This text is then formatted into a request and sent to the LLM for summarization. In this setup, documents uploaded by users are included in messages under the "user" role and sent to the Deepseek model, which returns summaries of the documents. Notably, all user inputs are processed under a single Deepseek account, which is typical in enterprise LLM deployments. However, this poses a privacy concern because Deepseek's prompt caching can inadvertently allow cached content to be reused across different users. As a result, a malicious user could potentially infer which documents other users have processed by exploiting timing side channels.

2) *Characterizing the Leakage*: We conducted experiments with 200 documents of various lengths (approximately 12,000, 18,000, and 24,000 tokens), each saved in PDF format and derived from segments of the zero Scrolls dataset [39]. Our results revealed distinct latencies between cache hits (where documents had been cached) and cache misses (where documents had not been cached), as shown in Figure 11. In particular, responses to cache hits remained consistently fast across different document lengths, while cache misses grew noticeably slower with larger documents.

3) *End-to-end Attacks*: In this evaluation, we assume an attacker aims to determine whether a specific document is uploaded by the victim for summarization. The attacker prepares a set of 200 documents of varying lengths (the "interested" documents). Meanwhile, the victim submits a total of 200 documents, half of which come from the interested set and half from outside it. This sequence is repeated 5 times, and each time the attacker attempts to distinguish which of the victim's documents belong to the interested set.

Specifically, the victim first submits 100 documents from the interested set. The attacker then probes each of the 200 interested documents once, recording response latencies. Based

TABLE IV
SUMMARY OF KV CACHE SHARING IN REAL WORLD
LLM SERVING SYSTEMS (DATE: 08/29/2024)

LLM service	System prompt sharing	User prompt sharing
GPT-4o-mini	✓	✓
Deepinfra	✓	✓
Deepseek-chat	✓	✓
Claude-3.5	✓	✓
Qwen-max	✗	✗
Moonshot	✓	✓
Baidu Ernie-8k	✗	✗
Google Gemini	✗	✗
Fireworks.ai	✓	✓
Groq	✗	✗
SiliconFLow	✓	✓

on a predefined threshold (2.0 seconds in our experiments), the TPR is computed as the fraction of probed documents correctly identified as cache hits (i.e., with latencies below the threshold). Next, the victim uploads 100 additional documents outside the interested set, and the attacker probes the entire set of 200 documents again. The FPR is then calculated as the fraction of documents incorrectly labeled as hits. Our evaluation shows that this attack achieves an average accuracy of 89%, with an average FPR of 0.05. A demo for the attack is presented on our website [35].

4) *Discussions*: Since KV-cache sharing typically requires requests to have a common prefix, we assume that the attacker knows either the full document or at least its prefix (i.e., the initial portion of the content). However, recent work has introduced more flexible cache-sharing mechanisms that extend beyond strict prefix matching. For example, CacheBlend [40] and LMCache [41] enable sharing of arbitrary reused text segments, while KVShare [42] and SemShareKV [43] support semantic-aware key-value cache sharing. These techniques are already being integrated into popular LLM serving engines such as vLLM, llm-d, and KServe. In such systems, an attacker could probe whether a target document has been processed by the LLM using only partial common text (not necessarily a prefix) or even general topical knowledge. We leave deeper exploration of these attack scenarios to future work.

D. Measurement Study on Commodity LLMs

1) *KV Cache Sharing*: To investigate KV cache sharing in commodity LLM services, we conducted experiments by invoking the APIs provided by these vendors. These APIs support different roles, such as system and user. For the measurement study, we designed requests with system and user prompts of varying lengths and configured them to run in the streaming mode. For this evaluation, we used the `zero_scroll` dataset for generating requests.

Specifically, we first measured the response latencies by sending initial requests that were likely to miss the cache. Then, we sent identical requests 10 times and measured the median latencies for these subsequent requests. To maximize the likelihood of co-locating on the same physical machine and ensuring the requests were cached, we conducted continuous

TABLE V
NATIVE SUPPORT OF SEMANTIC CACHING OF POPULAR
AI SERVICE PROVIDERS (DATE: 08/29/2024)

Service providers	Semantic cache support
Azure OpenAI Service models	✓
Amazon Bedrock	✓
Google Vertex AI	✗
Alibaba Elastic Algorithm Service (EAS) of Platform for AI (PAI)	✓

tests within the same time period. If we observed lower latencies in the later requests, this indicated the use of caching mechanisms in the LLM services. With KV cache sharing, the computation of matched prefix tokens during the prefill phase can be ignored. However, the output generated during the decoding phase still requires computation and inference, which are influenced by parameters such as temperature, introducing randomness. To verify that the latency reduction was due to KV cache sharing, we deliberately set a high temperature (0.9) in the request. This configuration was critical because semantic caching mechanisms typically return identical cached outputs for semantically similar inputs. By introducing substantial randomness in token selection through high temperature, we ensured the LLM would generate diverse responses despite input similarities. We verified whether the LLM produced different responses for each request, with TTFT reductions consistently observed. If it did, this strongly indicated that KV cache sharing was supported, enabling a reduction in TTFT while still allowing for diverse outputs. To minimize the impact of network latency, we sent requests of varying lengths, ranging from 200 to 2,000 tokens. The time difference between cached and uncached responses typically spanned several hundred milliseconds, making it easy to distinguish between the two. Additionally, we observed when the cache is hit, the TTFT remains consistent, regardless of the request length, whereas when the cache is missed, TTFT increases almost linearly as the length of the request grows. As summarized in Table. IV, most popular LLM service providers support KV cache sharing in specific scenarios.

2) *Semantic Cache Sharing*: We manually reviewed the documentation of public AI service providers to verify whether they support semantic cache APIs. As shown in Table. V, semantic caching is supported by major AI platform-as-a-service providers. Even on platforms that do not offer native semantic caching, users can still implement their own solutions or leverage open-source alternatives, such as GPTCache.

V. MITIGATIONS

A. Mitigating KV Cache Leakages

1) *Design*: A straightforward approach to mitigate KV cache leakages is to eliminate any sharing across requests. However, this would negate the computational and cost savings associated with caching. Noting that PSA recovers the request token by token by observing per-token timing differences, we explore the effect of a simple mitigation strategy that the prefix cache can only be shared in units of at least K tokens ($K = 2, 3, 4$ etc.). In SGLang, this could be achieved

TABLE VI
TOKEN RECOVERY RESULTS UNDER DIFFERENT NUMBERS
OF MINIMUM SHARED TOKENS

K	Recovery rate	Accuracy	#queries per recovered token	#queries per token
1	91.5%	97.9%	118.58	215.41
2	81.0%	98.8%	108.89	286.79
3	67.5%	98.5%	88.30	337.28
4	49.0%	98.0%	62.55	470.82

by modifying the radix tree structure used to manage the KV cache. This approach reduces the likelihood of KV cache sharing, but it is unlikely to significantly impact performance.

2) *Evaluation*: To evaluate the effectiveness of the mitigation and reduce the cost of querying the LLM, we conducted a simulation-based experiment. First, we built the classifiers that detect the hits and misses of K tokens for each value of K ($K = 1, 2, 3, 4$), following the method outlined in subsection IV-A. Since the timing differences become more pronounced as K increases, we reduced the number of samples (i.e., n) in multi-sampling, and obtained the corresponding TPRs and FPRs for each classifier. Then we used an *oracle* to simulate the classifiers by randomly sampling a number and determining whether it fell within the classification range. In this evaluation, we used the same repetitive trials method, dataset and fine-tuned model as described in subsection IV-A. The next-token predictor was modified to predict the next K tokens. Table VI presents the token recovery rate and the average number of queries per recovered token for $K = 1, 2, 3$ and 4. The results show that increasing K still yields a notable recovery rate, but the attack bottleneck shifts fundamentally. Larger K amplifies timing differences between cache hits and misses, reducing false positives. At the same time, the predictor's search space grows exponentially, making predictor accuracy the new bottleneck. For tokens that are successfully recovered, the average number of queries decreases with larger K values, since the predictor can more easily identify tokens in simpler prompts. However, the overall recovery rates decline because the predictor struggles with harder tokens at higher K . As a result, although queries per recovered token drop, the total cost of recovering an entire prompt rises significantly.

B. Mitigating Semantic Cache Leakages

1) *Design*: As investigated in subsection IV-B, private attributes have a significant impact on the semantic similarity between requests. As a result, the PNA infers private attributes by probing whether a semantically similar request is cached. To mitigate this leakage, we propose a strategy that involves identifying and anonymizing the private attributes present in the requests. This approach not only prevents the leakage of private attributes but also increases the potential for sharing requests across users.

As shown in Figure 12, we integrate a custom pre-processor and post-processor into the GPTCache framework. The pre-processor is designed to identify private attributes within the requests, and replace them with anonymized identifiers. In this approach, we selectively de-identify Personally Identifiable Information (PII) attributes, such as names, email addresses,

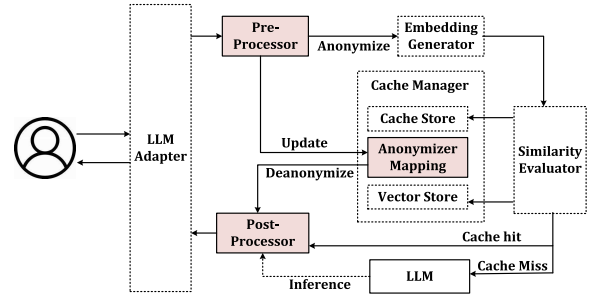


Fig. 12. Mitigating semantic cache leakages. The shaded components are customized as part of our mitigation.

phone numbers, credit card numbers, and IP addresses, while ensuring that no essential information needed for the LLMs is removed.

To facilitate reuse, the cache manager maintains a mapping structure that stores the anonymized identifier alongside its corresponding private attribute in a key-value format. The post-processor then identifies the anonymized identifiers in the response and replaces them with the private attributes by referencing this mapping. This ensures that the user receives an accurate response.

2) *Evaluation*: In our prototype implementation, we used the Presidio tool [44] to automatically identify private attributes. For performance evaluation, we used the evaluation dataset released by Presidio, which includes sentences containing private information. Specifically, we randomly sampled 1,000 sentences from the dataset and fed them into both the original GPTCache and the enhanced GPTCache. Then we measured the average delay introduced by the pre-processor and post-processor. The results show that the anonymization process adds an average delay of approximately 6 ms, while GPTCache's response latency for a semantic cache hit without anonymization is around 0.14 s. Thus, de-identification introduces only about 4% additional overhead, which has a minimal impact on GPTCache's overall performance.

VI. RELATED WORKS

1) Prompt Extraction Attacks With Adversarial Prompts:

Most existing research focuses on stealing system prompts from LLMs by eliciting the previous prompts, typically through direct output or translation. Earlier studies involved manually constructing attacker prompts [45], [46], while more recent work, such as PLeak, leverages output feedback from a given prompt and introduces an incremental search algorithm to optimize prompt retrieval [47]. While these approaches exploit the model's vulnerability to adversarial prompts, our proposed attacks take advantage of timing differences introduced in LLM service deployment. As such, our attacks do not rely on the specific details of any particular LLM.

Side channel attacks on LLM. DeBenedetti et al. propose system-level side channels within the deep learning lifecycle, such as training data filtering, input preprocessing, output monitoring, and query filtering. These side channels can potentially be exploited to infer the training data or the requests [48]. LLM keystroking attacks [49] are a type of packet analysis based side channels. These attacks exploit the length of

response tokens, assuming the attacker has access to encrypted network packets. Besides, Carlini and Nasr investigated the privacy risks of timing attacks in multi-turn interactions with LLM chatbots [50]. More recently, Soleimani et al. exposed network side-channel vulnerabilities in speculative decoding-optimized LLM services [51]. By comparison, we are the *first* to study the timing leaks introduced by LLM serving system optimizations, rather than relying on output data or token packet sizes to recover requests.

2) Micro-Architectural Side Channel Attacks On Deep Learning Systems:

Numerous studies have explored methods for extracting deep learning models or structures, as well as fingerprinting these models, by exploiting various side channels, such as CPU [52], [53], [54], [55], [56], GPU [57], FPGA [58], power and magnetic channels [59], [60], and PCIe traffic [61]. In comparison, our work focuses on leaking private prompts rather than stealing model parameters. Additionally, our approach does not rely on the micro-architectural or power characteristics of specific hardware; instead, it exploits timing leaks inherent in LLM systems. As a result, our attacks are applicable across CPU, GPU, and FPGA platforms, provided they utilize KV cache or semantic cache sharing techniques.

VII. CONCLUSION

LLM inference is a resource-intensive task, prompting studies focused on reducing inference latency. These optimizations often involve the use of various caches. When multiple users share the LLM system, these optimizations can lead to interference between users. This paper examines the side channels created by such interference, identifying two types of leaks: one in the KV cache and another in the semantic cache. We urge LLM system providers to recognize this emerging threat and prioritize security in their design choices.

REFERENCES

- [1] (2024). *Chatgpt — OpenAI*. [Online]. Available: <https://openai.com/chatgpt/>
- [2] (2024). *Perplexity AI*. [Online]. Available: <https://www.perplexity.ai/>
- [3] (2024). *Github Copilot · Your AI Pair Programmer*. [Online]. Available: <https://github.com/features/copilot/>
- [4] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27168–27183.
- [5] X. Wei et al., "Outlier suppression: Pushing the limit of low-bit transformer language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 17402–17414.
- [6] G. Xiao, J. Lin, M. Seznec, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 38087–38099.
- [7] W. Wang et al., "Model compression and efficient inference for large language models: A survey," 2024, *arXiv:2402.09748*.
- [8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 16344–16359.
- [9] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ. (SOSP)*, 2023, pp. 611–626.
- [10] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," 2023, *arXiv:2312.07104v2*.
- [11] (2024). *Prompt Template*. [Online]. Available: <https://python.langchain.com.cn/docs/modules/modelio/prompts/prompttemplates/>
- [12] (2024). *Prompt Caching With Claude*. [Online]. Available: <https://www.anthropic.com/news/prompt-caching>
- [13] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proc. 3rd Workshop Natural Lang. Process. Open Source Softw. (NLP-OSS)*, 2023, pp. 212–218.
- [14] (2024). *Langchain*. [Online]. Available: <https://www.langchain.com/>
- [15] G. Wu et al., "I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving," in *Proc. 32nd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2025, pp. 1–15.
- [16] X. Zheng et al., "InputSnatch: Stealing input in LLM services via timing side-channel attacks," 2024, *arXiv:2411.18191*.
- [17] C. Gu, X. L. Li, R. Kudithipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model APIs," 2025, *arXiv:2502.07776*.
- [18] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
- [19] (2024). *AnythingLLM: The All-in-One Desktop & Docker AI Application With Built-in Rag, AI Agents, and More*. [Online]. Available: <https://github.com/Mintplex-Labs/anything-llm>
- [20] M. Kurth, B. Gras, D. Andriesse, C. Giuffrida, H. Bos, and K. Razavi, "NetCAT: Practical cache attacks from the network," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 20–38.
- [21] M. Schwarzl, M. Schwarzl, M. Lipp, J. Masters, and D. Gruss, "NetSpectre: Read arbitrary memory over network," in *Proc. Eur. Symp. Res. Comput. Secur.*, Sep. 2019, pp. 279–299.
- [22] H. Naghibijouybari, A. Neupane, Z. Qian, and N. Abu-Ghazaleh, "Rendered insecure: GPU side channel attacks are practical," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 2139–2153.
- [23] Z. Zhou, W. Diao, X. Liu, Z. Li, K. Zhang, and R. Liu, "Vulnerable GPU memory management: Towards recovering raw data from GPU," 2016, *arXiv:1605.06610*.
- [24] H. Taneja, J. Z. Kim, J. Xu, S. V. Shaik, D. Genkin, and Y. Yarom, "Hot pixels: Frequency, power, and temperature attacks on GPUs and ARM SoCs," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 6275–6292.
- [25] *Samsung Bans Staff's AI Use After Spotting ChatGPT Data Leak*. Accessed: Mar. 15, 2025. [Online]. Available: <https://www.bloomberg.com/news/articles/2023-05-02/samsung-bans-chatgpt-and-other-generative-ai-use-by-staff-after-leak>
- [26] (2024). *System Prompts in Large Language Models*. [Online]. Available: <https://promptengineering.org/system-prompts-in-large-language-models/>
- [27] (2024). *Llamaindex is a Data Framework for Your LLM Applications*. [Online]. Available: <https://github.com/run-llama/llamaindex>
- [28] (2024). *Jpmorgan Rolls Out In-House GenAI-Based Chatbot To Employees*. [Online]. Available: <https://www.financedirectoreurope.com/news/jpmorgan-rolls-out-ai-based-chatbot/>
- [29] (2024). *Text Generation and Prompting*. [Online]. Available: <https://platform.openai.com/docs/guides/text?api-mode=responses>
- [30] L. Zheng et al., "Judging LLM-as-a-judge with MT-bench and chatbot arena," 2023, *arXiv:2306.05685*.
- [31] S. Wu et al., "BloombergGPT: A large language model for finance," 2023, *arXiv:2303.17564*.
- [32] (2024). *System Prompt Leakage Dataset*. [Online]. Available: <https://huggingface.co/datasets/gabrielchua/system-prompt-leakage>
- [33] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," 2018, *arXiv:1802.03426*.
- [34] (2024). *Flush Cache*. [Online]. Available: <https://github.com/sgl-project/sglang/blob/25e5d589e39b3b605296395e4f9c96ec42f09055/python/sglang/srt/server.py#L164>
- [35] (2025). *LLM Side-Channel Demo*. [Online]. Available: <https://sites.google.com/view/early-bird-catches-the-leak>
- [36] (2019). *Medquad*. [Online]. Available: <https://huggingface.co/datasets/lavita/MedQUAD>
- [37] (2024). *All-MPNET-Base-V2*. [Online]. Available: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>
- [38] (2024). *AI Document Summarization*. [Online]. Available: <https://www.ibm.com/architectures/hybrid/genai-document-summarization>
- [39] U. Shaham, M. Ivgi, A. Efrat, J. Berant, and O. Levy, "ZeroSCROLLS: A zero-shot benchmark for long text understanding," 2023, *arXiv:2305.14196*.
- [40] J. Yao et al., "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," 2024, *arXiv:2405.16444*.
- [41] (2025). *Lmcache: Supercharge Your LLM With the Fastest KV Cache Layer*. [Online]. Available: <https://github.com/LMCache/LMCache>

- [42] H. Yang et al., “KVShare: An LLM service system with efficient and effective multi-tenant KV cache reuse,” 2025, *arXiv:2503.16525*.
- [43] X. Zhao and S. Mastorakis, “SemShareKV: Efficient KVCache sharing for semantically similar prompts via token-level LSH matching,” 2025, *arXiv:2509.24832*.
- [44] (2022). *Presidio*. [Online]. Available: <https://github.com/microsoft/presidio>
- [45] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” 2022, *arXiv:2211.09527*.
- [46] Y. Zhang and D. Ippolito, “Prompts should not be seen as secrets: Systematically measuring prompt extraction attack success,” 2023, *arXiv:2307.06865*.
- [47] B. Hui, H. Yuan, N. Gong, P. Burlina, and Y. Cao, “PLeak: Prompt leaking attacks against large language model applications,” 2024, *arXiv:2405.06823*.
- [48] E. Debenedetti et al., “Privacy side channels in machine learning systems,” in *Proc. 33rd USENIX Secur. Symp.*, 2023, pp. 6831–6848.
- [49] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, “What was your prompt? A remote keylogging attack on AI assistants,” 2024, *arXiv:2403.09751*.
- [50] N. Carlini and M. Nasr, “Remote timing attacks on efficient language model inference,” 2024, *arXiv:2410.17175*.
- [51] M. Soleimani, G. Jia, I. Gim, S.-s. Lee, and A. Khandelwal, “Wiretapping LLMs: Network side-channel attacks on interactive LLM services,” *Cryptol. ePrint Arch.*, Paper 2025/167, 2025.
- [52] V. Duddu, D. Samanta, D. Vijay Rao, and V. E. Balas, “Stealing neural networks via timing side channels,” 2018, *arXiv:1812.11720*.
- [53] M. Yan, C. W. Fletcher, and J. Torrellas, “Cache telepathy: Leveraging shared resource attacks to learn DNN architectures,” in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 2003–2020.
- [54] A. S. Rakin, M. H. I. Chowdhury, F. Yao, and D. Fan, “DeepSteal: Advanced model extractions leveraging efficient weight stealing in memories,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 1157–1174.
- [55] C. Gongye, Y. Fei, and T. Wahl, “Reverse-engineering deep neural networks using floating-point timing side-channels,” in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
- [56] S. Shukla, M. Alam, P. Mitra, and D. Mukhopadhyay, “Stealing the invisible: Unveiling pre-trained CNN models through adversarial examples and timing side-channels,” 2024, *arXiv:2402.11953*.
- [57] J. Wei, Y. Zhang, Z. Zhou, Z. Li, and M. A. A. Faruque, “Leaky DNN: Stealing deep-learning model secret with GPU context-switching side-channel,” in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2020, pp. 125–137.
- [58] Y. Zhang, R. Yasaei, H. Chen, Z. Li, and M. A. A. Faruque, “Stealing neural network structure through remote FPGA side-channel analysis,” *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4377–4388, 2021.
- [59] H. T. Maia, C. Xiao, D. Li, E. Grinspun, and C. Zheng, “Can one hear the shape of a neural network?: Snooping the GPU via magnetic side channel,” in *Proc. USENIX Secur. Symp.*, 2021, pp. 4383–4400.
- [60] P. Horvath, L. Chmielewski, L. Weissbart, L. Batina, and Y. Yarom, “Barracuda: Bringing electromagnetic side channel into play to steal the weights of neural networks from NVIDIA GPUS,” 2023, *arXiv:2312.07783*.
- [61] Y. Zhu, Y. Cheng, H. Zhou, and Y. Lu, “Hermes attack: Steal DNN models with lossless inference accuracy,” in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 1973–1988.



Zixuan Pang is currently pursuing the master's degree with the University of Science and Technology of China. Her research interests include system security and confidential computing.



Wenhao Wang received the B.S. degree from the Ocean University of China in 2009 and the Ph.D. degree from the University of Chinese Academy of Sciences in 2015. He is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include system security, confidential computing, and side channels.



Zihao Wang received the Ph.D. degree from Indiana University Bloomington in 2025. He is currently a Research Fellow with Nanyang Technological University. His research interests include trustworthy machine learning, including safety, security, and privacy.



XiaoFeng Wang (Fellow, IEEE) received the Ph.D. degree in computer engineering from Carnegie Mellon University in 2004. His research interests include cloud and mobile security and data and health informatics security. He is a fellow of ACM and AAAS. He is currently the Chair of the ACM Special Interest Group on Security, Audit and Control (SIGSAC) and was also the TPC Co-Chair of the ACM Conference on Computer and Communications Security (CCS), the ACM's flagship security and privacy conference, from 2018 to 2019.



Linke Song is currently pursuing the Ph.D. degree with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include system security and confidential computing.



Hongbo Chen received the bachelor's degree from Xi'an Jiaotong University in 2018. He is currently pursuing the Ph.D. degree with Indiana University Bloomington. His research interests include system security research, especially on AI for security and confidential computing.



Wei Song (Senior Member, IEEE) received the Ph.D. degree in computer science from The University of Manchester, Manchester, U.K., in 2011. He is currently an Associate Professor with the Institute of Information Engineering, CAS. His current research interests include the security enhancement of computer architectures, such as the defenses for cache side channel and control-flow hijacking attacks.

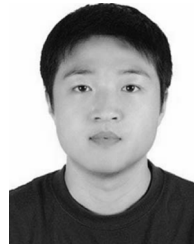


Dan Meng received the B.S., M.S., and Ph.D. degrees from Harbin Institute of Technology, Harbin, Heilongjiang, China. His research interests include high performance computer architecture and cyber security.



Yier Jin (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from Yale University in 2012. He is currently a Professor with the University of Science and Technology of China and an Adjacent Professor with the University of Florida. His research interests include hardware security, embedded systems design and security, trusted hardware intellectual property (IP) cores, hardware software co-design for modern computing systems, security analysis on the Internet of Things (IoT), and wearable devices with particular emphasis

on information integrity and privacy protection in the IoT era. He was a recipient of the DoE Early CAREER Award in 2016 and the ONR Young Investigator Award in 2019.



Rui Hou received the B.S. and M.S. degrees in computer architecture from Harbin Institute of Technology, China, in 2001 and 2003, respectively, and the Ph.D. degree in computer architecture from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, China, in 2007. He is currently a Professor and the Director of the State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences. He has published more than 40 papers in international conferences and

journals, and has more than 50 patents. His current research interests include computer architecture, processor security, data center server architecture, and AI security.